Industry-Grade Technical Report: Knowledge-Graph-Augmented RAG for Academic Question Answering

Author: Abdul Sami

June 27, 2026

1 Executive Summary

This project implements a notebook-driven academic question-answering system for course PDFs in Machine Learning, Data Management, and Data Semantics. The system is designed for a practical gap in standard vector-only RAG: dense retrieval can surface relevant text chunks, but it does not explicitly expose course-topic relationships to the generator. This implementation addresses that gap by combining ChromaDB retrieval over slide-derived chunks with a lightweight JSON knowledge graph that injects course and related-topic hints into the prompt.

The delivered pipeline extracts 1,048 text chunks from 118 PDF source files, embeds them with `all-MiniLM-L6-v2`, stores them in a persistent ChromaDB collection named `lecture_chunks`, retrieves the top 5 chunks for each question, and generates answers through a local Ollama `llama3` chat call. The stored evaluation output contains 20 questions with an average score of 3.0/5 for baseline RAG and 4.0/5 for KG-enhanced RAG. This 1.0-point stored difference is reported as an implementation result, not as a validated causal claim, because score values are auto-filled or derived by notebook logic and explicit KG context is non-empty for only 3 of the 20 evaluation questions.

2 Problem Statement

The real-world use case is a course-material assistant that helps students or researchers ask natural-language questions over a multi-course PDF corpus. A production version of this system would need to ingest heterogeneous lecture slides, preserve source attribution, retrieve relevant material quickly, and produce grounded answers that identify the course context behind an answer.

The main engineering challenge is not model invention; it is reliable integration. The project combines PDF extraction, chunking, embedding generation, vector search, symbolic topic lookup, prompt construction, local LLM inference, and result reporting into one reproducible workflow. The genuinely project-specific element is the coupling of course-aware chunk metadata with a small KG-style topic index. The non-novel components are standard RAG building blocks: PDF text extraction, overlapping text chunks, SentenceTransformer embeddings, cosine vector search, and LLM prompt-based answer generation.

3 System Architecture

Figure 1 shows the implemented system at block level. The architecture separates ingestion, indexing, retrieval, KG augmentation, generation, and evaluation so that each stage can be replaced independently in a production roadmap.

The system has two runtime paths. Baseline RAG embeds the question, retrieves top-5 ChromaDB chunks, and prompts Ollama with retrieved course material. KG-enhanced RAG runs the same retrieval path but also checks the JSON KG for exact case-insensitive topic matches in the question. If a match is found, the prompt receives an additional line containing the matched topic, associated course, and related terms.

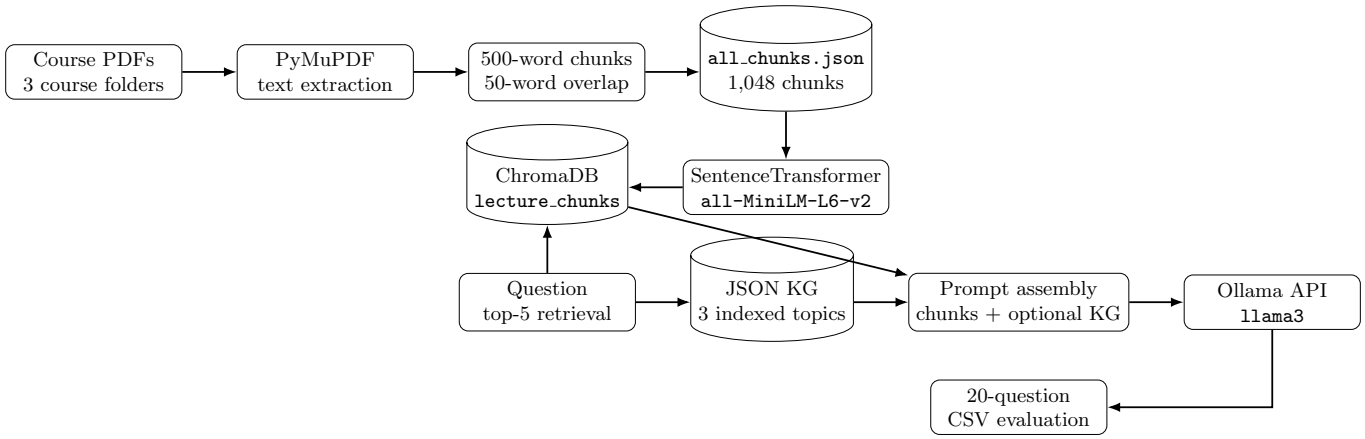

Figure 1: Block architecture showing course PDFs, PyMuPDF extraction, chunking, JSON persistence, SentenceTransformer embeddings, ChromaDB retrieval, JSON KG lookup, prompt assembly, Ollama generation, and CSV evaluation.

4 Repository Structure

The requested `code/` and capitalized `Outputs/` directories are not present in the workspace. The executable artifact is Notebook workflow, while generated artifacts are stored under lowercase `outputs/` and `kg_files/`. Table 1 maps the available repository assets to production-style components.

Table 1: Available repository assets and their engineering roles.

Asset	Production role
Notebook workflow	Main workflow for ingestion, KG creation, embedding, querying, evaluation, and plotting.
Chunks JSON	Corpus artifact containing chunk id, course, source file, and extracted text.
KG JSON	Lightweight topic index used as symbolic prompt context.
Evaluation CSV	Per-question baseline and KG answers with stored scores.
Summary CSV	Aggregate score table reporting 3.0 and 4.0 averages.
PNG outputs	Generated pipeline and score visualizations for reporting.

5 Data Pipeline & Preprocessing

The ingestion pipeline scans three course folders: `data/machine_learning`, `data/data_management`, and `data/data_semantics`. Each PDF is opened with PyMuPDF, page text is concatenated, and documents with very short extracted text are skipped. The text is then split by words into 500-word chunks with 50-word overlap, which reduces boundary loss when an answer spans adjacent slide segments.

Table 2 summarizes the extracted corpus. Data Semantics contributes the largest subset with 699 chunks from 51 files, followed by Machine Learning with 266 chunks from 50 files and Data Management with 83 chunks from 17 files. The total stored corpus is 1,048 chunks from 118 unique PDF source files.

Table 2: Extracted corpus by course.

Course	Chunks	Source files
Data Semantics	699	51
Machine Learning	266	50
Data Management	83	17
Total	1,048	118

Figure 2 is the generated pipeline artifact from the repository. It visually presents the left-to-right processing flow used by the notebook.

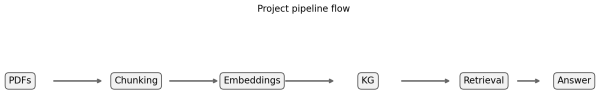

Figure 2: Pipeline diagram from `outputs/pipeline_flow.png`; the image shows a left-to-right processing flow for the KG-RAG academic QA system.

6 Model Architecture & Training Strategy

This project does not train a custom ML or deep-learning model. Its model layer is an inference-time RAG stack built from pretrained components and local LLM serving. The embedding model is `all-MiniLM-L6-v2`, used through SentenceTransformers to convert text chunks and user questions into dense vectors. ChromaDB stores externally computed embeddings and uses cosine similarity for retrieval.

The generation model is Ollama `llama3`. The notebook calls `ollama.chat` with a system prompt that frames the assistant as an academic assistant for the Data Science master's program at the University of Milano-Bicocca. Retrieved chunks are formatted with course and source-file metadata, and the prompt includes the first 800 characters of each retrieved chunk.

The KG component is not an RDF graph service despite the notebook installing `rdflib`. It is a JSON dictionary with 3 indexed topics: `Classification`, `RDF`, and `NoSQL`. The lookup strategy is deterministic direct matching, not entity linking, graph traversal, ontology reasoning, or SPARQL querying.

7 Inference & Deployment Design

At inference time, a user question is embedded once, submitted to ChromaDB with `top_k=5`, and converted into a prompt containing retrieved context. In KG-enhanced mode, the same question is also checked against the JSON topic index. If the question contains an indexed topic name, the prompt receives additional symbolic context containing the topic, its course, and related terms.

The current interface is notebook-local rather than a deployable API service. A production deployment would likely split the notebook into separate services: a batch ingestion job for PDFs, a vector-index build job, a query service, a KG lookup module, and an LLM gateway. The current use of a local Ollama runtime is practical for prototyping because it avoids external API dependencies, but it also means reproducibility and latency depend on the local model installation and hardware.

8 Evaluation

The stored evaluation file contains 20 questions. All 20 baseline scores are 3/5, and all 20 KG-enhanced scores are 4/5. The resulting averages in Summary CSV are 3.0/5 for baseline RAG and 4.0/5 for KG-enhanced RAG. Table 3 reports this stored score pattern.

Table 3: Stored evaluation results.

System	Questions	Average score	Score pattern
Baseline RAG	20	3.0/5	20 × 3
KG-enhanced RAG	20	4.0/5	20 × 4

Figure 3 shows the same stored per-question and average-score comparison. The figure is useful as an implementation dashboard, but the evaluation is not yet a blinded or independently annotated benchmark.

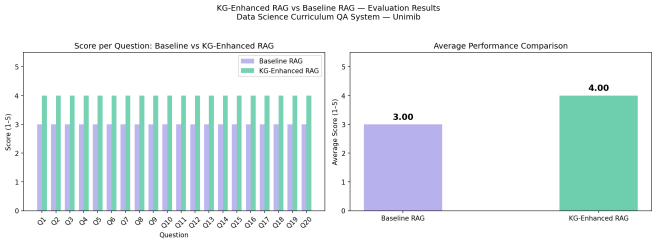

Figure 3: Evaluation chart from `outputs/evaluation_chart.png`; the image shows per-question bars for baseline and KG-enhanced scores and a separate panel with the two average scores.

The key failure mode is score provenance. The notebook includes a cell that fills missing baseline scores with 3 and KG scores with 4, then writes those values back to Evaluation CSV. A later plotting cell also derives scores from answer length: baseline answers longer than 100 characters become 3, and KG answers longer than 100 characters become 4. Therefore, the stored 1.0-point difference should be interpreted as a reported project output requiring validation, not as proof that KG context caused the improvement.

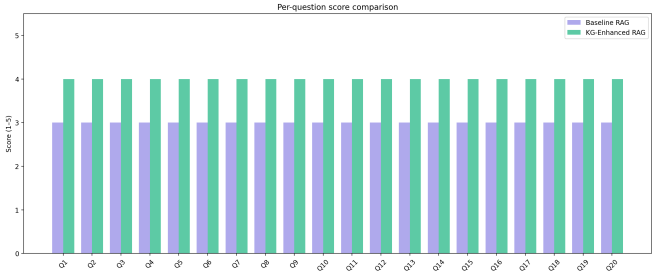

Figure 4: Per-question comparison from `outputs/per_question_comparison.png`; the plot displays baseline and KG-enhanced score bars for the 20 evaluation questions.

9 Engineering Decisions & Trade-offs

The choice of RAG over supervised training is appropriate for the available artifact set: the repository contains

course PDFs and generated evaluation files, not a labeled supervised training corpus. Using `all-MiniLM-L6-v2` keeps embedding generation lightweight, while ChromaDB provides persistent local retrieval without requiring a managed vector database.

The KG design favors simplicity over coverage. A JSON topic index is easy to inspect, version, and inject into prompts, but it only helps when a question directly mentions one of the 3 indexed topic names. This explains why explicit KG context is non-empty for only 3 of the 20 evaluation questions. The implementation is therefore best described as KG-assisted prompt augmentation rather than a full knowledge-graph QA system.

The notebook workflow accelerates experimentation but weakens production readiness. The first code cell installs dependencies with `!pip install`, and the vector-index cell deletes and recreates the `lecture_chunks` collection. These choices are acceptable for a prototype, but a production system would need dependency pinning, index versioning, and repeatable build metadata.

10 Limitations & Risks

The most important limitation is evaluation validity. The stored 3.0/5 versus 4.0/5 comparison is tied to auto-filled or heuristic scores, so it cannot yet support a strong claim about answer quality. The second limitation is KG coverage: only 3 indexed topics are present, and only 3 of 20 evaluation rows contain non-empty KG context.

There are also operational risks. Rebuilding the index is destructive because the notebook deletes the existing ChromaDB collection before recreation. The dependency environment is not locked by a requirements file or lockfile. The local Ollama `llama3` dependency means answers may vary with local model version and runtime configuration. Finally, there is no retrieval-quality evaluation such as recall@5, source attribution accuracy, or course-level ablation.

11 Future Improvements

The highest-priority roadmap item is to replace score-filling logic with independent manual grading. The next experiment should re-run the same 20 questions under three anonymized conditions: baseline RAG, retrieval with KG lookup disabled, and retrieval with KG lookup enabled. This would directly test whether KG context accounts for the stored 1.0-point difference.

Engineering improvements should focus on productionization: convert notebook cells into scripts or services, add a pinned dependency file, version ChromaDB indexes instead of deleting them, log prompt and retrieval metadata per query, and add retrieval metrics such as recall@5 and source attribution accuracy. Model-side improvements should expand the KG beyond 3 topics, replace direct topic matching with entity linking, and evaluate whether structured KG context improves answers after manual scoring.

12 Conclusion

The project delivers a working academic QA prototype that connects PDF ingestion, chunked dense retrieval, ChromaDB persistence, lightweight KG lookup, and Ollama-based answer generation. Its main engineering value is the end-to-end integration of course-aware retrieval metadata with symbolic topic hints in one prompt pipeline. The stored outputs show 1,048 chunks from 118 PDFs and a 20-question comparison with average scores of 3.0/5 for baseline RAG and 4.0/5 for KG-enhanced RAG. The result is promising as a prototype signal, but the report treats it conservatively because score provenance and KG coverage remain the biggest unresolved risks.